

زورخانه همیشه جای آرامی بود.

صدای نفس ها، ضرب آهنگ تمرین، و پهلوانانی که سال ها با یک قانون نانوشته زندگی کرده بودند: قدرت باید کنترل شود. تا آن روز. روزی که موجودی عجیب وارد این دنیا شد؛ یک لاکپشت بزرگ، سخنگو، با رفتاری که نه شبیه مردم این سرزمین بود و نه شبیه هیچ چیزی که تا آن روز دیده بودند.

می خندید، شوخی می کرد و بی خبر از ترسی که ایجاد کرده بود، میان مردم می چرخید. پهلوانان که مسئول حفظ نظم بودند، او را خطری ناشناخته دیدند و دستگیرش کردند. نه از روی بدخواهی، بلکه از روی ناآگاهی.

اما این پایان ماجرا نبود.

کمی بعد، سه لاکپشت دیگر به همراه استادشان که یک موش پیر بود وارد دنیای پهلوانان شدند. حالا دیگر سرنوشت این بیگانگان با مبارزات تن به تن مشخص میشد

صفی، مفرد و یاور، هر کدام روبه روی یکی از لاکپشت ها ایستادند.

در این مبارزات، قدرت به تنهایی کافی نبود.

فاصله، زمان، واکنش و تصمیم گیری نقش اصلی را داشتند.

هیچ کس نمی دانست نتیجه چه خواهد شد. همه چیز به انتخاب ها بستگی داشت. پل اتصال به تمرین

حالا برای فهمیدن نتیجه ی این مبارزات، ما قصد داریم در این پروژه مبارزات را شبیه سازی کنیم.

در این شبیه سازی، هر مبارز یک عامل تصمیم گیر است که در هر لحظه باید انتخاب کند: نزدیک شود یا فاصله بگیرد؟ حمله کند یا صبر کند؟ ریسک کند یا محتاط باشد؟

شما با طراحی این تصمیم ها و شبیه سازی مبارزات، مسیر داستان را ادامه می دهید و مشخص می کنید که این رویارویی ها چگونه به پایان می رسند.



راهنمای بازی Fighter



مقدمه

در بازی *Fighter*، دو Agent در مقابل یکدیگر قرار می‌گیرند که هرکدام دارای 100 Hitpoint هستند. هنگامی که Hitpoint یکی از Agent ها به صفر برسد، بازی پایان می‌یابد و آن Agent شکست می‌خورد. بازی به طور مداوم برنامه‌ی Agent شما را اجرا کرده و وضعیت فعلی بازی را به آن ارائه می‌دهد، سپس نتیجه عملیات انتخابی شما را از برنامه گرفته و اجرا می‌کند.

تعامل با بازی

کد Agent شما باید تابعی داشته باشد که وضعیت فعلی بازی را به عنوان ورودی دریافت کرده و عملیاتی که قصد انجام آن را دارید به عنوان خروجی ارائه دهد. این ورودی‌ها و خروجی‌ها به صورت JSON (در پایتون، این داده‌ها به صورت dict هستند) منتقل می‌شوند. بازی هر بار ورودی‌های جدید را از شما دریافت کرده و پس از پردازش، آن‌ها را در بازی اعمال می‌کند.



ساختار ورودی ها:

در هر فریم، اطلاعات وضعیت بازی به تابع شما به صورت دو مجموعه داده زیر ارسال می شود:

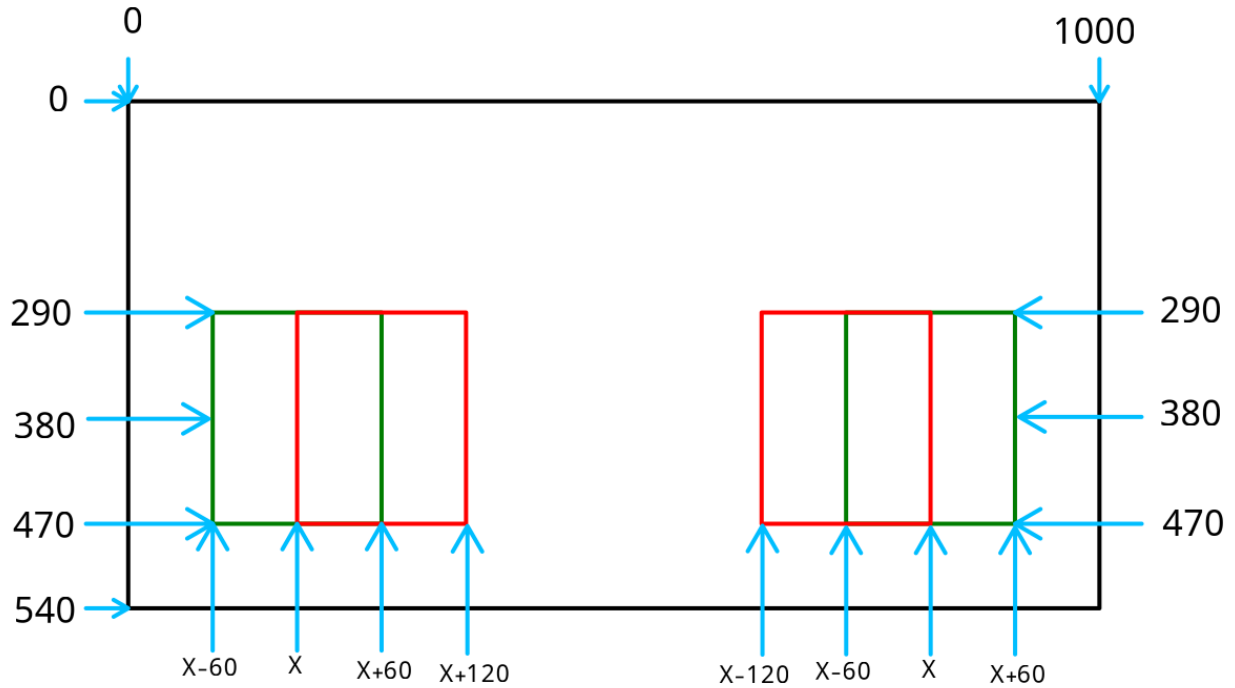
1. **fighter_info** – اطلاعات مربوط به Agent شما.
2. **opponent_info** – اطلاعات مربوط به حریف شما.
3. **saved_data** – داده های ذخیره شده که برای حفظ وضعیت های خاص در طول بازی استفاده می شود.

مقادیر fighter_info به شرح زیر هستند:

```
fighter_info = {  
    "x": int,          // شما Agent مکان افقی.  
    "y": int,          // شما Agent مکان عمودی.  
    "health": int,     // (Hitpoint) مقدار سلامت 0 تا 100.  
    "attacking": boolean, // آیا در این لحظه مشغول حمله کردن هستید؟  
    "attack_cooldown": [int, int], // حمله های شما cooldown زمان.  
    "jump": boolean,   // آیا در حال پرش هستید؟  
    "dash_cooldown": int // dash. برای استفاده مجدد از قابلیت cooldown زمان.  
}
```

مقادیر opponent_info نیز به شرح زیر هستند:

```
opponent_info = {  
    "x": int,          // مکان افقی حریف شما.  
    "y": int,          // مکان عمودی حریف شما.  
    "health": int,     // حریف (Hitpoint) مقدار سلامت.  
    "attacking": boolean // آیا حریف شما در حال حمله است؟  
}
```



توضیحات مختصات:

- مختصات x و y مربوط به مرکز hitbox شما است.
- ابعاد hitbox شما مستطیلی به عرض 120 و طول 180 است.
- صفحه بازی دارای طول 1000 و عرض 540 است.
- هنگام ایستادن، مقدار y شما برابر با 380 است، ولی در هنگام پرش این مقدار می تواند تا 170 کاهش یابد.
- برد حمله شما به صورت مستطیلی عمودی به عرض 120 و طول 180 است که از مرکز hitbox شما شروع می شود.

نکته مهم: اگر فاصله افقی شما با حریف دقیقاً 180 واحد باشد، ضربه شما به حریف اصابت نخواهد کرد. برای برخورد با حریف، فاصله باید کمتر از 180 باشد.



خروجی مورد نیاز:

شما باید یک دیکشنری شامل موارد زیر برگردانید:

```
action = {  
    "move": string | null,    // (در صورت عدم حرکت null یا "left" یا "right" حرکت به سمت  
    "attack": int | null,    // (Heavy یا 2 برای حمله سنگین (Light Attack) مقدار حمله: 1 برای حمله سبک  
    "jump": bool,            // آیا قصد پرش دارید؟  
    "dash": string | null,    // (در صورت عدم استفاده null یا "left" یا "right" به سمت Dash حرکت  
    "debug": any,            // اطلاعات دیباگ در صورت نیاز به نمایش  
    "saved_data": dict | json // داده های ذخیره شده که باید به روزرسانی شوند  
}
```

شرح عملگرها و ویژگی ها:

- **حمله سبک (Light Attack):** این نوع حمله 10 damage به حریف وارد می کند و نیاز به 25 فریم cooldown دارد.
- **حمله سنگین (Heavy Attack):** این نوع حمله 20 damage به حریف وارد می کند و نیاز به 100 فریم cooldown دارد.
- **حرکت:** حرکت به سمت راست یا چپ در هر فریم 5 واحد تغییر مکان ایجاد می کند.
- **Dash:** در طول 10 فریم، شما را 300 واحد به سمت راست یا چپ جابجا می کند و پس از اتمام آن، 40 فریم cooldown دارد.

نکات مهم:

- در هر فریم، باید تا قبل از 0.4 ثانیه تصمیمات خود را ارسال کنید، در غیر این صورت فریم از دست خواهد رفت.
- اگر کد شما باعث crash شدن بازی شود، شما بازنده اعلام می شوید.
- در صورتی که نیاز به اطلاعات بیشتری برای دیباگ دارید، می توانید آن ها را در قسمت debug قرار دهید.
- اگر مایل به استفاده از پایتون هستید agent.py را تغییر داده و GAMECODE-python.py را اجرا کنید



چگونه حریف رو شکست بدیم؟

شما برای حل پروژه بالا باید یک درخت Min-Max برای انتخاب حرکت و یک تابع هیوریستیک برای ارزیابی وضعیت های بازی طراحی کنید. این دو بخش در کنار هم به Agent اجازه می دهند با در نظر گرفتن وضعیت فعلی و پیش بینی محدود آینده، بهترین تصمیم را در هر فریم بگیرد.

مهم ترین بخش یک Agent هوشمند، توانایی آن در ارزیابی وضعیت بازی است. شما نمی توانید تمام حرکات ممکن آینده را تا انتهای بازی پیش بینی کنید؛ به همین دلیل، به یک تابع ارزیابی نیاز دارید که به هر وضعیت خاص از بازی یک امتیاز اختصاص دهد و مشخص کند آن وضعیت تا چه اندازه برای Agent مطلوب است. یک تابع هیوریستیک خوب، ترکیبی از چندین پارامتر کلیدی بازی است. برای مثال، امتیاز یک وضعیت می تواند به صورت زیر محاسبه شود:

$$\text{Score} = (w1 * \text{health_diff}) - (w2 * \text{distance_to_opponent}) + (w3 * \text{cooldown_advantage}) + \dots$$

در این تابع، معیارهایی مانند اختلاف سلامتی بین شما و حریف، فاصله تا حریف (برای حمله یا فرار)، وضعیت Cooldown و آماده بودن حملات، موقعیت مکانی روی صفحه (مثلاً قرار گرفتن در گوشه)، و وضعیت حمله حریف نقش اساسی دارند. با تنظیم وزن ها ($w1$ ، $w2$ و ...) می توانید تعیین کنید کدام عامل در تصمیم گیری Agent اهمیت بیشتری دارد و در نهایت به بهترین ترکیب برای ارزیابی وضعیت بازی برسید.

پس از تعریف تابع هیوریستیک، از آن در درخت Min-Max استفاده می شود. در این درخت، Agent حرکات ممکن خود و پاسخ های احتمالی حریف را تا عمق محدودی شبیه سازی می کند و در گره های انتهایی، وضعیت ها را با استفاده از تابع هیوریستیک امتیازدهی می کند. نتیجه این فرآیند، انتخاب حرکتی است که در بدترین حالت نیز بهترین نتیجه را برای Agent تضمین کند.

برای کاهش پیچیدگی تصمیم گیری، به جای کار مستقیم با حرکات سطح پایین در هر فریم، می توان از حرکات با انتزاع بالا (High-Abstract actions) استفاده کرد. این حرکات رفتار کلی Agent را در یک بازه زمانی مشخص می کنند و سپس به مجموعه ای از حرکات پایه ای در فریم های متوالی تبدیل می شوند. نمونه هایی از این حرکات با انتزاع بالا عبارت اند از:

- فرار از دشمن تا زمانی که Cooldown حمله شما تمام شود.
- حمله کردن به حریف، زمانی که حمله شما آماده است.
- عقب نشینی و حفظ فاصله، زمانی که سلامتی شما کم است.



- انجام یک حمله سنگین، زمانی که فرصت مناسبی پیش می آید.
- استفاده از Dash برای غافلگیر کردن حریف و نزدیک شدن سریع به او.

این حرکات با انتزاع بالا می توانند به عنوان اعمال اصلی در درخت Min-Max استفاده شوند و در نهایت به تصمیم های لحظه ای و سطح پایین در هر فریم تبدیل شوند. این رویکرد باعث می شود هم تصمیم گیری Agent ساختارمندتر شود و هم هزینه محاسباتی جستجو کاهش یابد.